

Merge sort

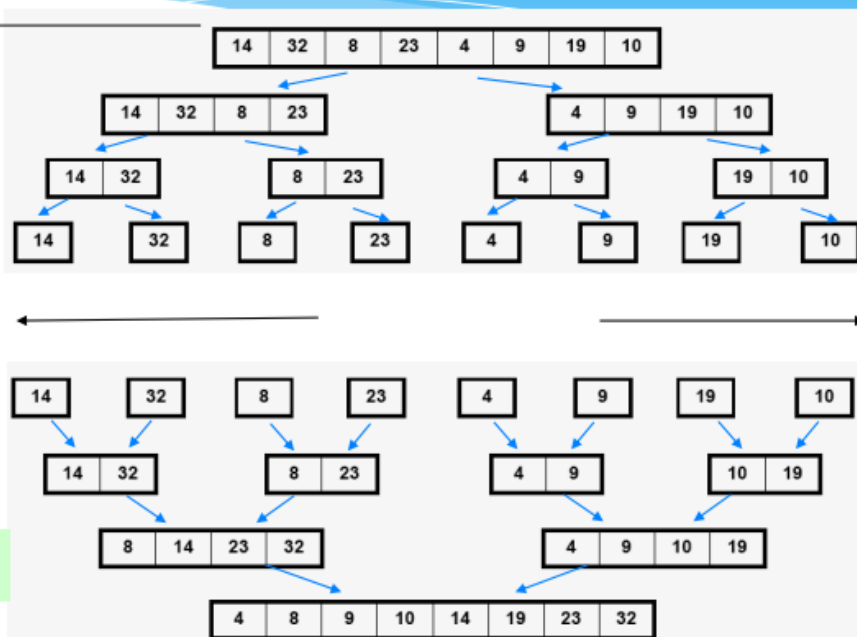
- Merge sort is based on the **divide-and-conquer** paradigm
- Conceptually, a merge sort works as follows:
 1. Divide the unsorted list into n sub lists, each containing 1 element (a list of 1 element is considered sorted)
 2. Repeatedly merge sub lists to produce new sorted sub lists until there is only 1 sub list remaining. This will be the sorted list

Merge sort

Input

How much work at every step?

How much work at every step?



$O(\lg n)$ steps

$O(\lg n)$ steps

Merge sort

```
void MergeSort(int *A,int n) {
    int mid, i, *L, *R;

    if(n < 2)
        return; // base condition. If the array has less than two element, do nothing.

    mid = n/2; // find the mid index.

    // create left and right subarrays
    // mid elements (from index 0 till mid-1) should be part of left sub-array
    // and (n-mid) elements (from mid to n-1) will be part of right sub-array

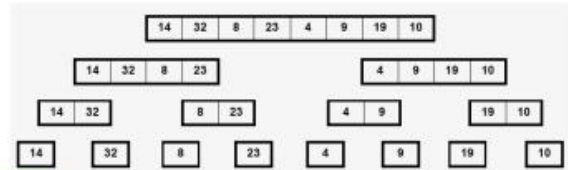
    L = (int*)malloc(mid*sizeof(int));
    R = (int*)malloc((n - mid)*sizeof(int));

    for(i = 0;i<mid;i++)
        L[i] = A[i]; // creating left subarray

    for(i = mid;i<n;i++)
        R[i-mid] = A[i]; // creating right subarray

    MergeSort(L,mid); // sorting the left subarray
    MergeSort(R,n-mid); // sorting the right subarray

    Merge(A,L,mid,R,n-mid); // Merging L and R into A as sorted list.
}
```



Merging

L:

3	10	23	54
---	----	----	----

 R:

1	5	25	75
---	---	----	----

Result:

1	3	5	10	23	25	54	75
---	---	---	----	----	----	----	----

Merging

L:

3	10	23	54
---	----	----	----

 R:

1	5	25	75
---	---	----	----

$i = 0$ $j = 0$

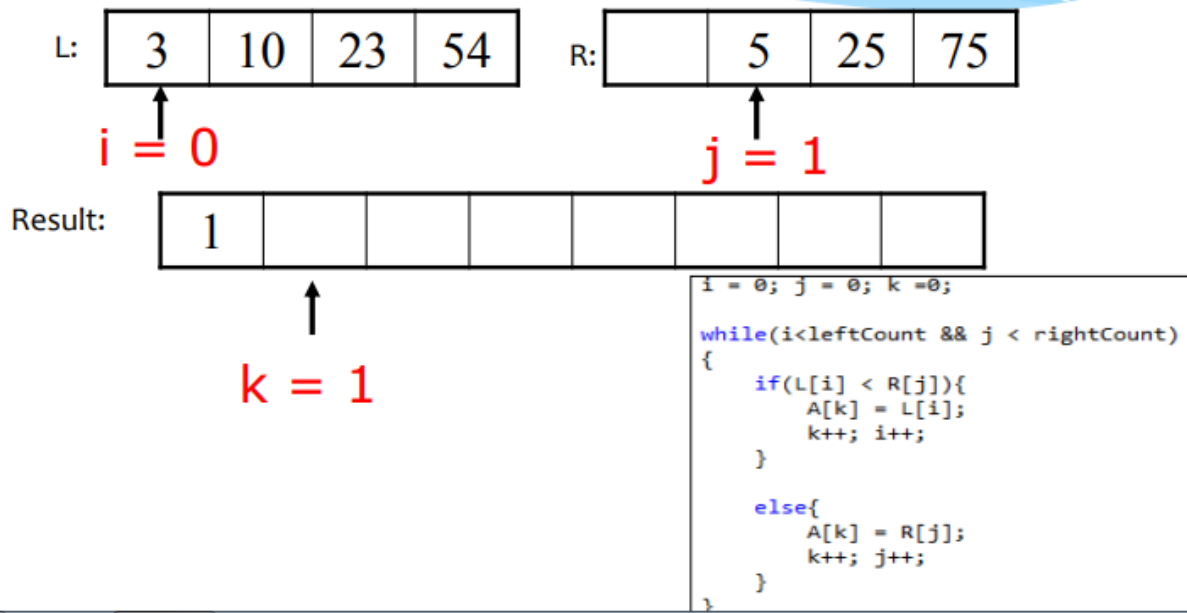
Result:

--	--	--	--	--	--	--	--

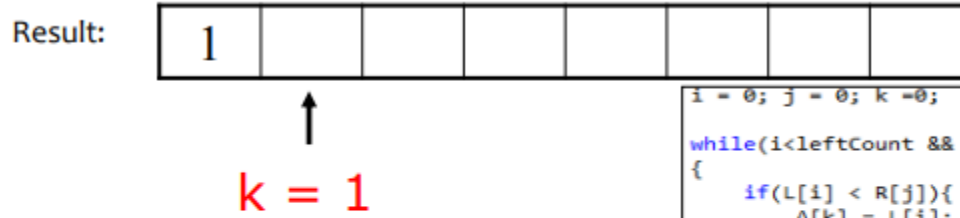
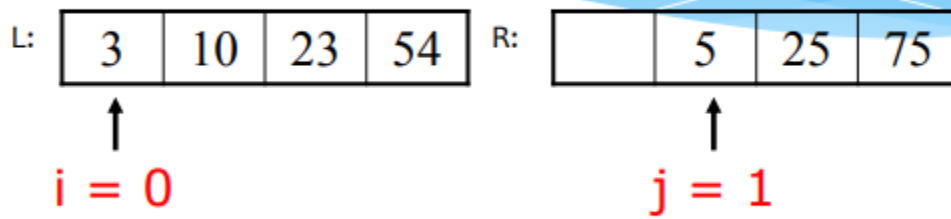
$k = 0$

```
i = 0; j = 0; k = 0;
while(i < leftCount && j < rightCount)
{
    if(L[i] < R[j]){
        A[k] = L[i];
        k++; i++;
    }
    else{
        A[k] = R[j];
        k++; j++;
    }
}
```

Merging

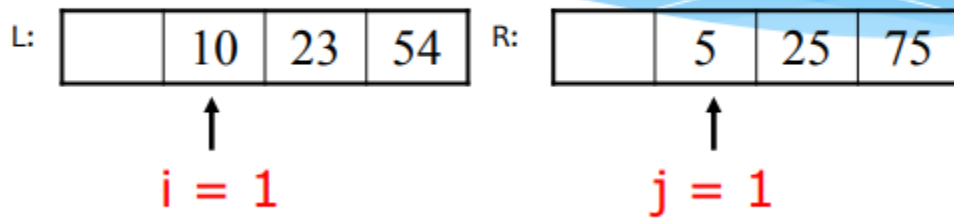


Merging

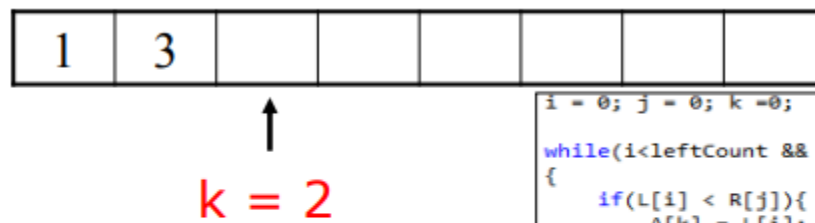


```
i = 0; j = 0; k = 0;
while(i < leftCount && j < rightCount)
{
    if(L[i] < R[j]){
        A[k] = L[i];
        k++; i++;
    }
    else{
        A[k] = R[j];
        k++; j++;
    }
}
```

Merging

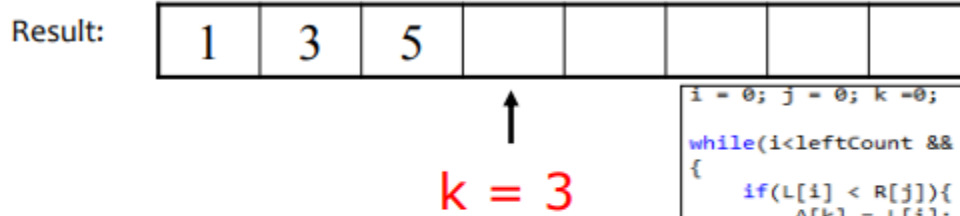
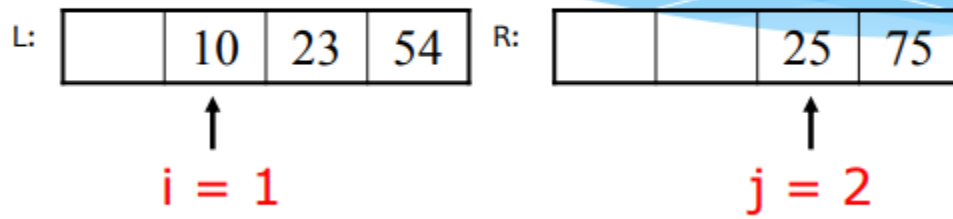


Result:



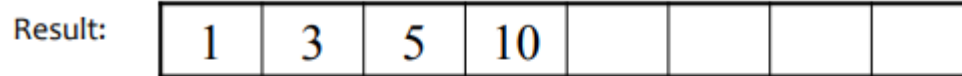
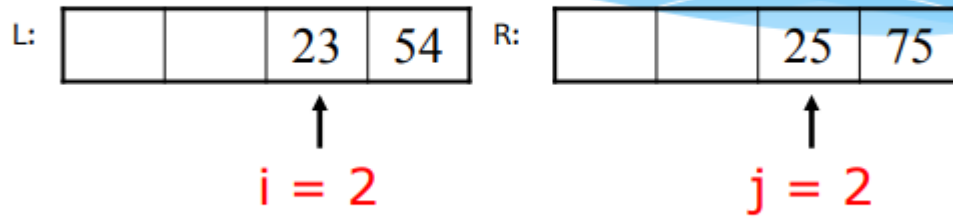
```
i = 0; j = 0; k = 0;
while(i < leftCount && j < rightCount)
{
    if(L[i] < R[j]){
        A[k] = L[i];
        k++; i++;
    }
    else{
        A[k] = R[j];
        k++; j++;
    }
}
```

Merging



```
i = 0; j = 0; k = 0;
while(i < leftCount && j < rightCount)
{
    if(L[i] < R[j]){
        A[k] = L[i];
        k++; i++;
    }
    else{
        A[k] = R[j];
        k++; j++;
    }
}
```

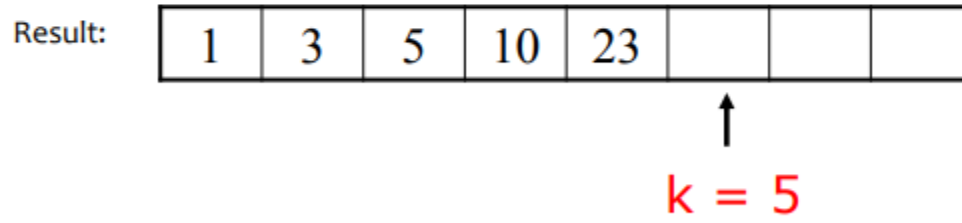
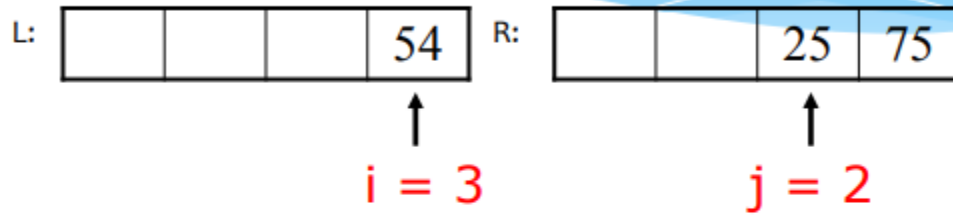
Merging



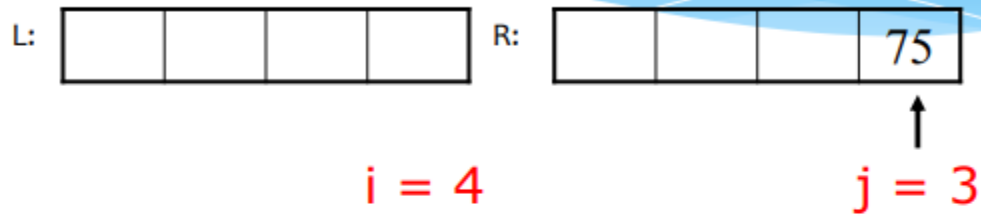
$k = 4$

```
i = 0; j = 0; k = 0;
while(i < leftCount && j < rightCount)
{
    if(L[i] < R[j]){
        A[k] = L[i];
        k++; i++;
    }
    else{
        A[k] = R[j];
        k++; j++;
    }
}
```

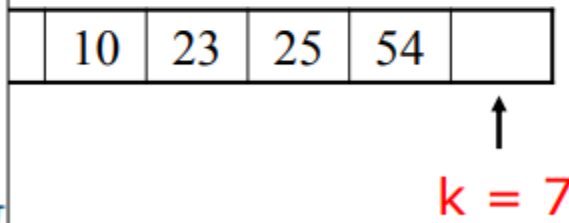

Merging



Merging



```
while(i < leftCount){  
    A[k] = L[i];  
    k++; i++;  
}  
  
while(j < rightCount){  
    A[k] = R[j];  
    k++; j++;  
}
```



Merging

L:

--	--	--	--

R:

--	--	--	--

$i = 4$

$j = 4$

Result:

1	3	5	10	23	25	54	75
---	---	---	----	----	----	----	----



$k = 7$

Merge sort: program

```
void MergeSort(int *A,int n) {
    int mid, i, *L, *R;

    if(n < 2)
        return; // base condition. If the array has less than two element, do nothing.

    mid = n/2; // find the mid index.

    // create left and right subarrays
    // mid elements (from index 0 till mid-1) should be part of left sub-array
    // and (n-mid) elements (from mid to n-1) will be part of right sub-array

    L = (int*)malloc(mid*sizeof(int));
    R = (int*)malloc((n - mid)*sizeof(int));

    for(i = 0;i<mid;i++)
        L[i] = A[i]; // creating left subarray

    for(i = mid;i<n;i++)
        R[i-mid] = A[i]; // creating right subarray

    MergeSort(L,mid); // sorting the left subarray
    MergeSort(R,n-mid); // sorting the right subarray

    Merge(A,L,mid,R,n-mid); // Merging L and R into A as sorted list.
}
```

Merge sort: program

```
void Merge(int *A,int *L,int leftCount,int *R,int rightCount) {
    int i,j,k;

    i = 0; j = 0; k =0;

    while(i<leftCount && j < rightCount)
    {
        if(L[i] < R[j]){
            A[k] = L[i];
            k++; i++;
        }

        else{
            A[k] = R[j];
            k++; j++;
        }
    }

    while(i < leftCount){
        A[k] = L[i];
        k++; i++;
    }

    while(j < rightCount){
        A[k] = R[j];
        k++; j++;
    }
}
```